OSEI TUTU DICKSON - 8563421
ALLEN ELORM AHANOGBE - 8530121

1. AI-BASED MENTAL HEALTH SYSTEM

2. PROJECT PROPOSAL

1. Introduction

Mental health challenges have become increasingly prevalent in modern society, with millions of individuals facing issues such as anxiety, depression and stress. Despite growing awareness, access to mental health support remains limited due to stigma, cost and availability of professionals. Advances in artificial intelligence (AI) present an opportunity to bridge this gap by providing scalable, accessible and personalized mental health solutions. This project proposes the development of an AI-based mental health system featuring an intelligent chatbot for immediate advice and a counselor booking platform to connect users with licensed professionals. By integrating these components, the system aims to offer a comprehensive, user-friendly approach to mental health care.

2. Problem Statement

Many individuals lack timely access to mental health resources. Traditional therapy can be expensive, time-consuming and geographically restrictive, while self-help tools often lack personalization or professional oversight. Additionally, people experiencing distress may hesitate to seek help due to stigma or uncertainty about where to start. Existing digital mental health solutions often provide either chatbot-based support or counselor services, but rarely both in an integrated, seamless manner. This creates a gap in delivering continuous, holistic care that combines immediate AI-driven guidance with human expertise when needed.

3. Aim of Project

The aim of this project is to design and develop an AI-based mental health system that empowers users to manage their mental well-being through two key features:

(1) an AI chatbot providing real-time advice and emotional support, and
(2) a platform for scheduling appointments with licensed counselors. The system seeks to improve accessibility, reduce barriers to seeking help and offer a hybrid approach to mental health care.

4. Specific Objectives

1. To develop an AI chatbot capable of understanding user emotions, providing evidence-based mental health advice and escalating cases requiring professional intervention.

2. To create a user-friendly interface for booking virtual or in-person appointments with certified counselors.

3. To ensure the system adheres to privacy and ethical standards, including data security and confidentiality.

4. To integrate natural language processing (NLP) and machine learning (ML) to personalize chatbot interactions based on user input and history.

5. To evaluate the system's effectiveness through user feedback and mental health outcome metrics.

5. Justification

The need for innovative mental health solutions is evident from rising global mental health statistics and the strain on traditional healthcare systems. The World Health Organization (WHO) estimates that one in four people will experience a mental health issue at some point in their lives, yet treatment gaps persist, especially in low-resource settings. An AI-based system can provide 24/7 support, reduce costs and reach underserved populations. By combining AI-driven advice with human counselors, the project addresses both immediate needs and long-term care, offering a balanced and scalable solution.

6. Significance of Project

This project has the potential to transform how mental health support is accessed and delivered. It will:

1. Increase accessibility by providing initial support via the chatbot.

2. Reduce stigma by offering a private, non-judgmental platform for seeking help.

3. Enhance efficiency by triaging users-offering self-help tools to those with mild issues and connecting those with severe needs to counselors.

4. Contribute to the growing field of digital health by demonstrating the efficacy of hybrid AI-human mental health interventions.

7. Motivation

The motivation for this project stems from the urgent need to address the mental health crisis exacerbated by factors such as the COVID-19 pandemic, social isolation, and economic instability. Personal stories of individuals struggling to find timely support, combined with the transformative potential of AI, inspire this initiative. The vision is to create a tool that not only supports users in distress but also empowers them to take control of their mental well-being with confidence and ease.

8. Scope of Project

The project will focus on developing a functional prototype of the AI-based mental health system. Key features include:

1. chatbot with conversational AI capabilities for mental health support (e.g., mood tracking, coping strategies, crisis detection).

2. A counselor booking module with a database of licensed professionals, appointment scheduling and virtual consultation options.

3. A web-based application targeting English-speaking users initially.

4. Basic personalization and progress tracking for users. The scope excludes physical health diagnostics, in-person therapy logistics and multilingual support, which may be considered in future iterations.

9. Methodology

The project will adopt an iterative development approach, combining AI research, software engineering and user-centered design principles. Phases include requirement analysis, system design, development, testing and evaluation.

Development Tools

5. **AI/Chatbot Development**: Python, PyTorch (for ML models), Natural Language Toolkit (NLTK).
6. **Frontend**: React (web interface).
7. **Backend**: Node.js or Django, PostgreSQL (database for counselor and user data).
8. **APIs**: Twilio (for notifications), Zoom API (for virtual counseling).
9. **Security**: Encryption tools (e.g., AES-256), compliance with HIPAA/GDPR standards.

Plan of Activities

1. **Month 1-2**: Research mental health frameworks (e.g., CBT, DBT), define system requirements and gather user needs through surveys.
2. **Month 3-4**: Design system architecture, train AI chatbot with mental health datasets, and build counselor database.
3. **Month 5-6**: Develop frontend and backend, integrate chatbot and booking system.
4. **Month 7**: Conduct alpha testing with simulated users, refine features based on feedback.
5. **Month 8**: Beta testing with real users, ensure compliance with privacy laws.
6. **Month 9**: Finalize system, prepare documentation and deploy prototype.

10. Project Deliverables

- A fully functional AI chatbot capable of providing mental health advice and detecting escalation needs.
- A counselor booking platform with an operational scheduling system and virtual consultation integration.
- Web application with an intuitive user interface.
- Technical documentation, including system architecture, codebase, and user manual.
- A report on user testing results, including feedback and system performance metrics.
- A presentation summarizing the project's development, outcomes and future recommendation.

3 . REQUIREMENTS DOCUMENT

3.1 Functional User Requirements

These specify what the system must allow users to do.

UR1: User Registration and Login

- ➢ Users must be able to create an account using an email and password.
- ➢ Users must log in securely to access personalized features.
- ➢ Option for "Remember Me" to simplify future logins.

UR2: Chatbot Interaction

- ➢ Users can interact with an AI chatbot for real-time support or to ask mental health-related questions.
- ➢ Users can end the chat session at any time.

UR3: Personalized Recommendations

- ➢ Users can view tailored mental health resources (e.g., articles, exercises, videos) based on their interaction with the chatbot.
- ➢ Users can save or bookmark resources for later access.

·**UR4: Privacy and Settings**

- ➢ Users can manage their account settings (e.g., update email, password).
- ➢ Users can opt-in or out of data sharing for system improvement.

·**UR5: Booking Counselors**

- ➢ Users can view a list of available licensed counselors with their specialties and availability.
- ➢ Users can book an appointment with a counselor by selecting a date and time slot.
- ➢ Users receive a confirmation email and can cancel or reschedule appointments.

3.2 Functional System Requirements

These define the system's internal functionality to support user requirements.

SR1: User Authentication

- The system must securely store and encrypt user credentials.
- The system must support session management and timeout after inactivity.

·**SR2: Chatbot Functionality**

- The system must integrate an AI chatbot capable of natural language processing (NLP) to respond to user queries.
- The system must log chatbot interactions for quality improvement (anonymized unless opted in).
- **SR3: Resource Database**

SR4: Security and Compliance

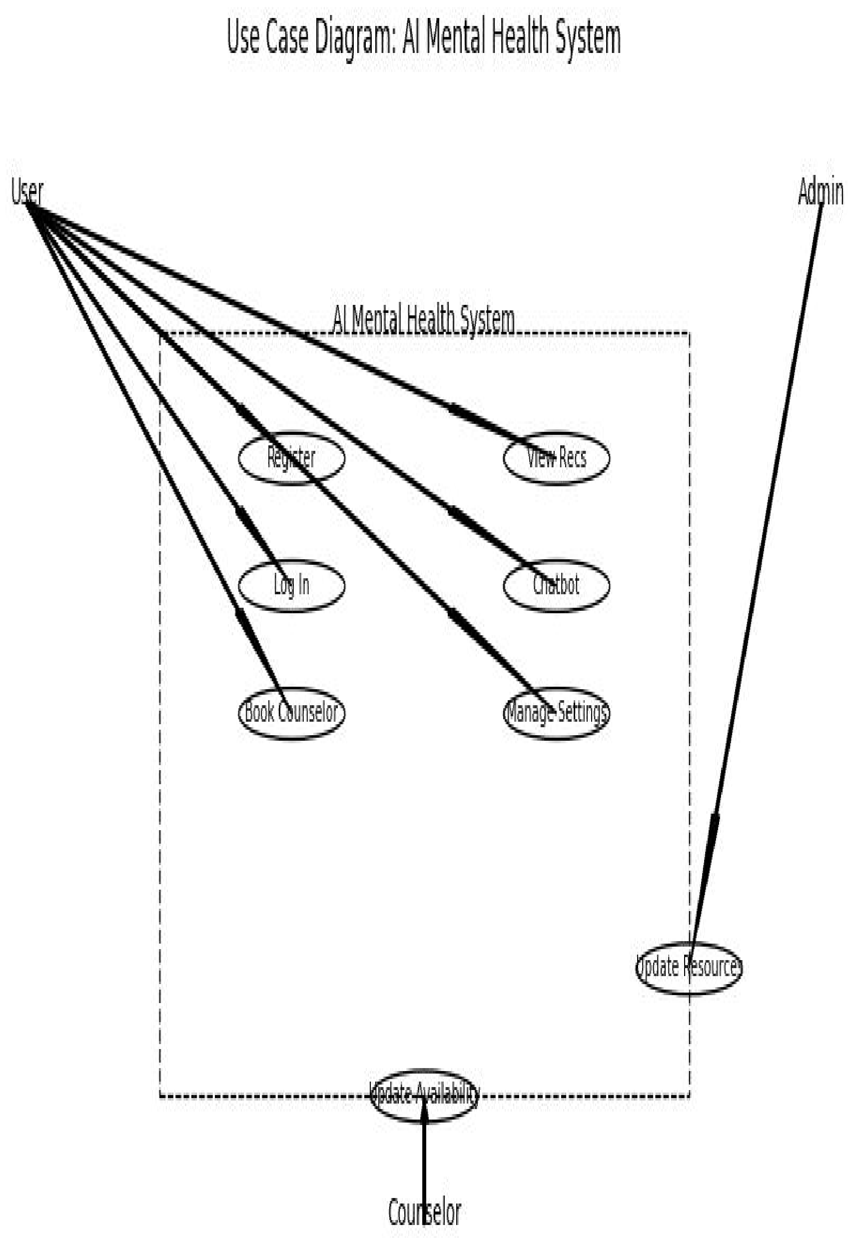
- The system must comply with data privacy regulations (e.g., GDPR, HIPAA if applicable).
- The system must encrypt sensitive user data both in transit and at rest.

SR7: Counsellor Booking System

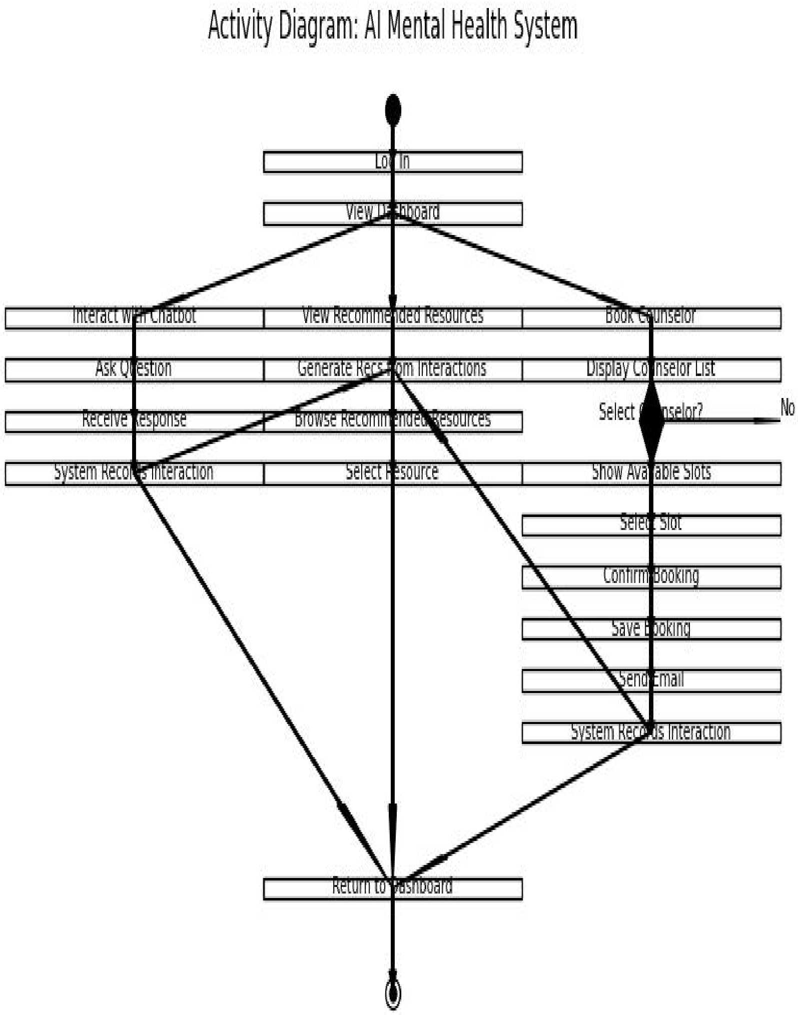
- The system must maintain a database of counselors, including their profiles, specialties and availability schedules.
- The system must manage appointment bookings, ensuring no double-booking occurs.
- The system must send confirmation emails to users and notifications to counselors upon booking, cancellation, or rescheduling.

3.2 UML Diagrams

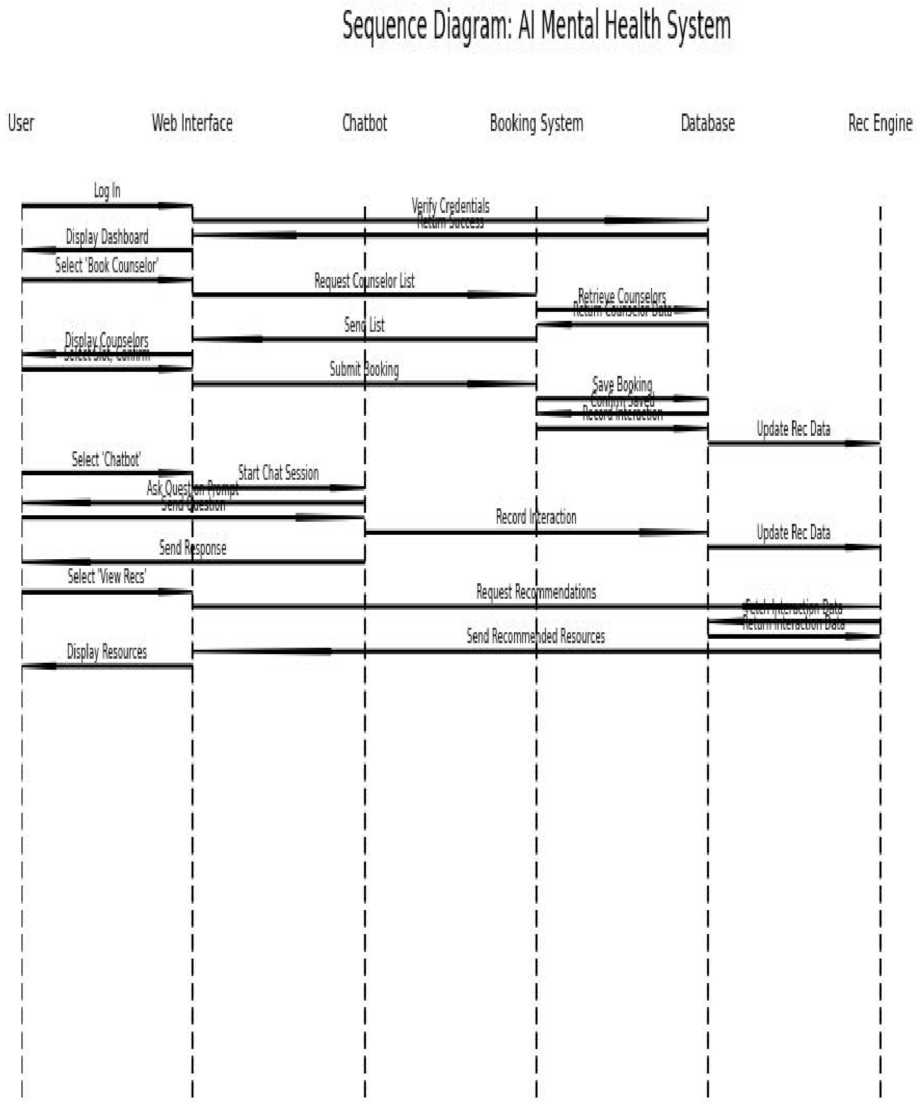
3.2.a Use Case Diagram



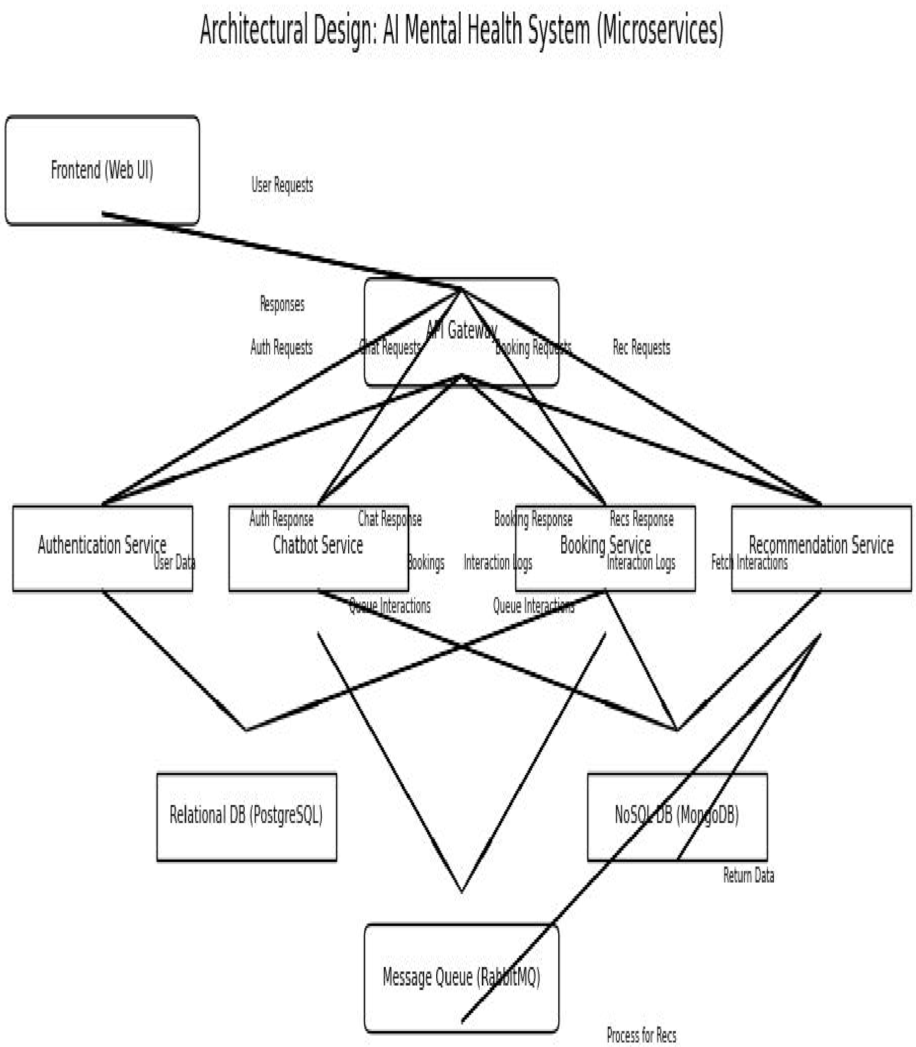
3.2.b Activity Diagram



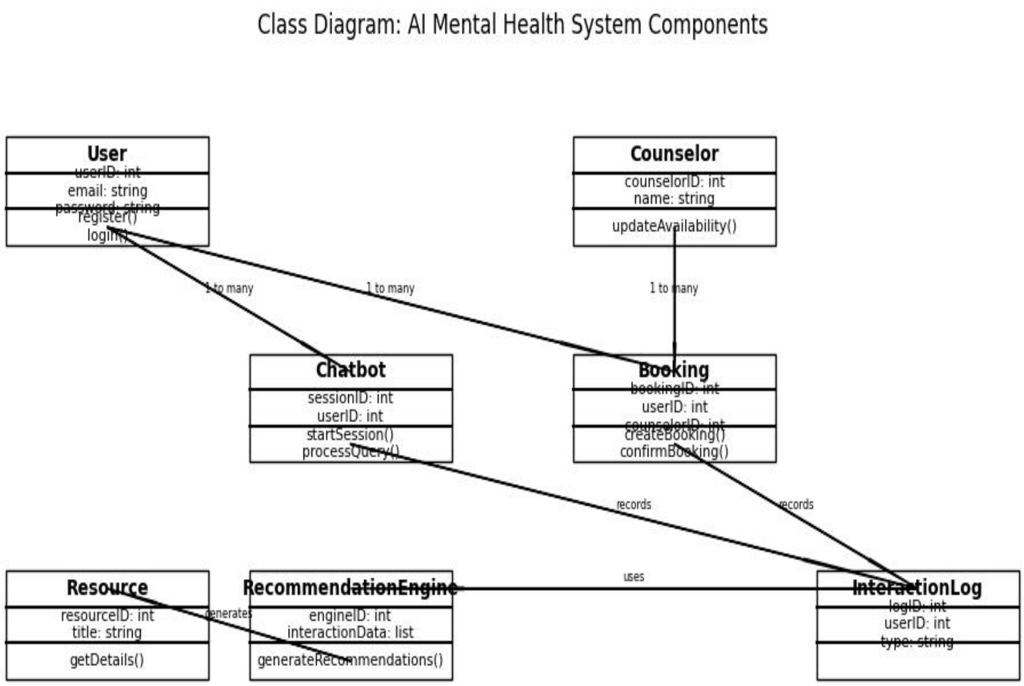
3.2.c Sequence Diagram



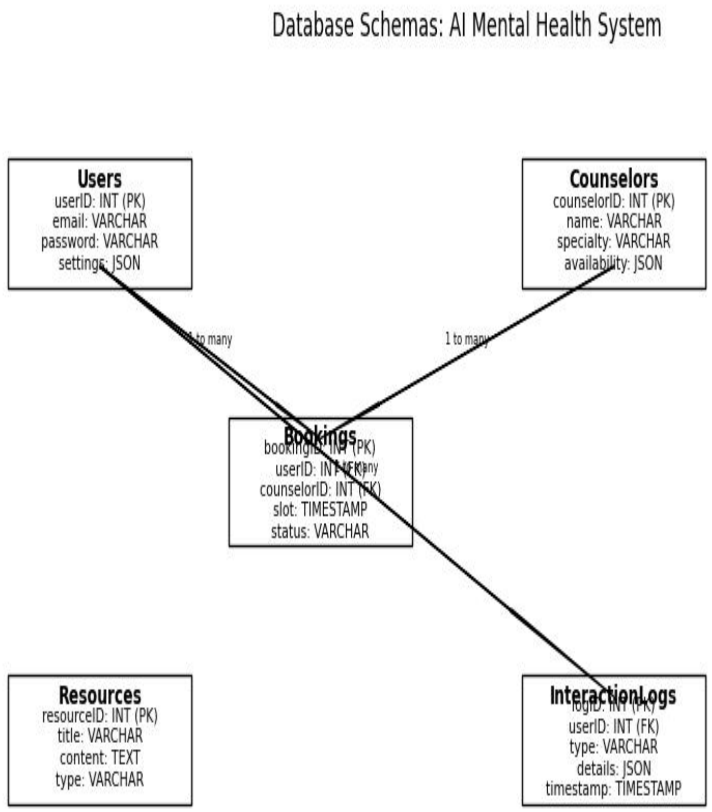
4. PROJECT DESIGN
4.1 Architectural Design



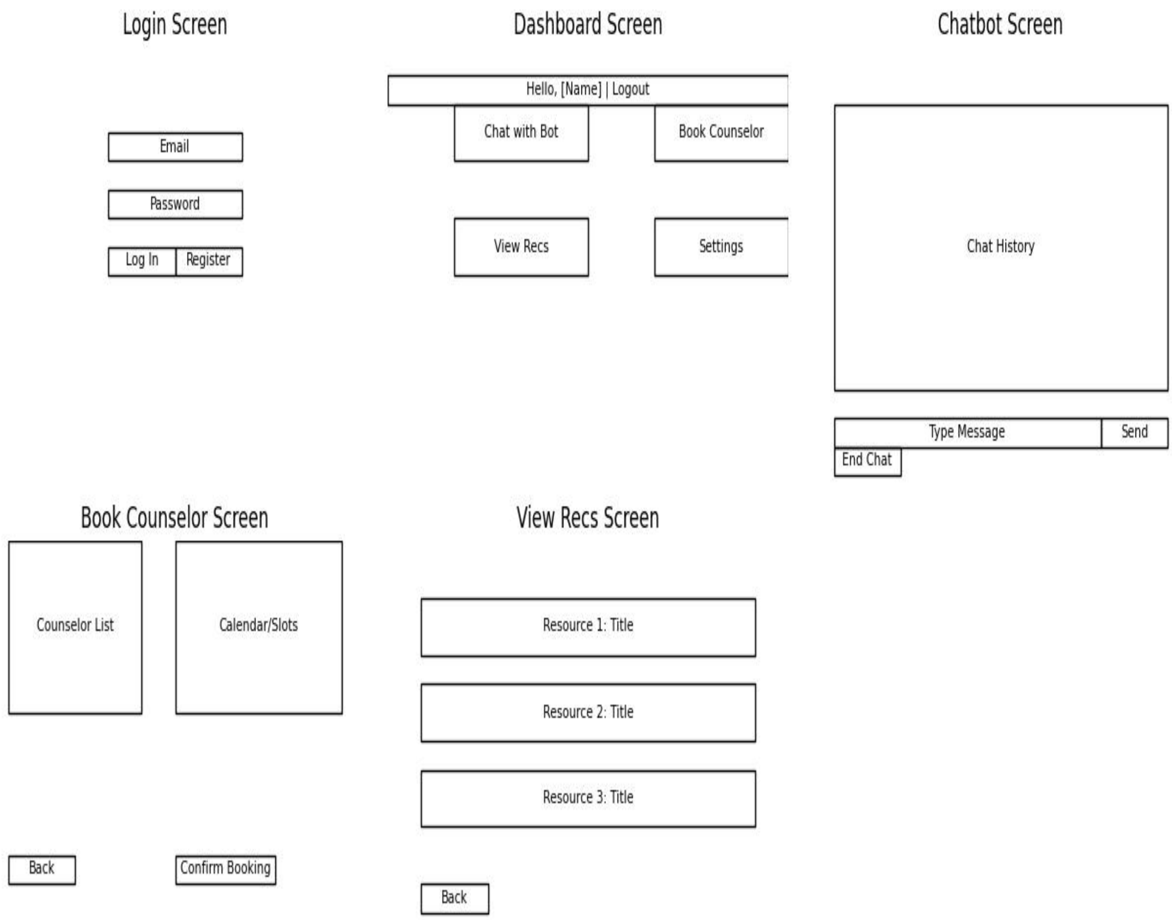
4.2 Components Design
- Class Diagram



4.3 Database Schemas



4.4 UI Design
- Wire frames of the User interface



5. Provide the Algorithm(s) for your program

Algorithm 1: User Login

```
Algorithm UserLogin(email, password)
    Input: email (string), password (string)
    Output: Boolean (success/failure), userID (int) if successful

    1. Retrieve userRecord from Users table where email = input_email
    2. If userRecord is null:
            Return False, null
    3. storedPassword = userRecord.password
    4. If hash(password) equals storedPassword:
            userID = userRecord.userID
            Return True, userID
       Else:
            Return False, null
```

Algorithm 2: Chatbot Interaction

```
Algorithm ChatbotInteraction(userID, query)
    Input: userID (int), query (string)
    Output: response (string)

    1. sessionID = GenerateUniqueSessionID()
    2. response = ProcessQuery(query)    // Uses NLP model (e.g., BERT) to
generate response
    3. interactionDetails = { "query": query, "response": response }
    4. timestamp = GetCurrentTimestamp()
    5. logEntry = { logID: GenerateUniqueLogID(), userID: userID, type: "chatbot",
                    details: interactionDetails, timestamp: timestamp }
    6. Insert logEntry into InteractionLogs table
    7. UpdateRecommendationEngine(userID, logEntry)
    8. Return response

Function ProcessQuery(query)
    Input: query (string)
    Output: response (string)
    1. tokenizedQuery = Tokenize(query)
    2. response = NLPModel Predict(tokenizedQuery)    // e.g., pre-trained chatbot
model
    3. Return response
```

Algorithm 3: Book Counselor Appointment

```
Algorithm BookCounselorAppointment(userID, counselorID, slot)
    Input: userID (int), counselorID (int), slot (datetime)
    Output: Boolean (success/failure), bookingID (int) if successful

    1.  Retrieve counselorRecord from Counselors table where counselorID =
    input_counselorID
```

```
2. If counselorRecord is null:
        Return False, null
3. availability = counselorRecord.availability
4. If slot not in availability:
        Return False, null
5. bookingID = GenerateUniqueBookingID()
6. bookingRecord = { bookingID: bookingID, userID: userID, counselorID: counselorID,
                     slot: slot, status: "confirmed" }
7. Insert bookingRecord into Bookings table
8. Update availability in counselorRecord by removing slot
9. Update Counselors table with new availability
10. interactionDetails = { "counselorID": counselorID, "slot": slot }
11. timestamp = GetCurrentTimestamp()
12. logEntry = { logID: GenerateUniqueLogID(), userID: userID, type: "booking",
                 details: interactionDetails, timestamp: timestamp }
13. Insert logEntry into InteractionLogs table
14. UpdateRecommendationEngine(userID, logEntry)
15. SendConfirmationEmail(userID, bookingRecord)
16. Return True, bookingID
```

Algorithm 4: Generate Recommended Resources

```
Algorithm GenerateRecommendedResources(userID)
    Input: userID (int)
    Output: resources (list of Resource objects)

    1. interactionLogs = Retrieve all records from InteractionLogs where userID = input_userID
    2. If interactionLogs is empty:
            Return defaultResources    // Predefined list of general resources
    3. interactionData = ExtractDetails(interactionLogs)    // e.g., queries, booking types
    4. keywords = AnalyzeInteractions(interactionData)    // Extract key terms (e.g., "stress", "anxiety")
    5. resources = []
    6. For each keyword in keywords:
            matchingResources = Retrieve from Resources table where title or content contains keyword
            resources = resources + matchingResources
    7. rankedResources = RankResources(resources, interactionData)    // Based on relevance
    8. Return rankedResources[0:5]    // Return top 5 resources

Function AnalyzeInteractions(interactionData)
    Input: interactionData (list of log details)
    Output: keywords (list of strings)
    1. keywords = []
```

```
        2. For each log in interactionData:
                If log.type = "chatbot":
                        keywords = keywords + ExtractKeywords(log.details.query)
                If log.type = "booking":
                        keywords = keywords +
ExtractKeywords(log.details.counselorID.specialty)
        3. Return unique(keywords)

Function RankResources(resources, interactionData)
        Input: resources (list), interactionData (list)
        Output: rankedResources (list)
        1. scores = {}
        2. For each resource in resources:
                score = CalculateRelevance(resource, interactionData)    // e.g., keyword
frequency
                scores[resource] = score
        3. rankedResources = Sort resources by scores descending
        4. Return rankedResources
```

6. Draw the Flowchats for your algorithms (Use Lucidchat software or any other flowchat software)